

▼ PART 1 — Understand the Business Problem

Your main KPI is:

Customer Satisfaction Score (CSS) = Average Product Rating.

So our central question is:

What factors are influencing Sephora's Customer Satisfaction Score?

We will investigate:

Brand ↓ Product ↓ Category ↓ Price ↓ Reviews ↓ Recommendations ↓ Review Helpfulness ↓ Customer Characteristics ↓ Customer Satisfaction Score

```
!pip install gdown
```

```
Requirement already satisfied: gdown in /usr/local/lib/python3.12/dist-packages (5.2.2)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from gdown) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from gdown) (3.29.7)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.12/dist-packages (from gdown) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from gdown) (4.67.3)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->gdown) (2.8.4)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->gdown) (4.12.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (2025.1.1)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (1.7.1)
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

▼ PART 2 — Load the Data

Double-click (or enter) to edit

```
products = pd.read_csv("products.csv", low_memory=False)
reviews = pd.read_csv("reviews.csv", low_memory=False)
```

```
-----
ParserError                                Traceback (most recent call last)
/tmp/ipykernel_1652/3369108398.py in <cell line: 0>()
      1 products = pd.read_csv("products.csv", low_memory=False)
----> 2 reviews = pd.read_csv("reviews.csv", low_memory=False)
```

```
-----
3 frames -----
/usr/local/lib/python3.13/dist-packages/pandas/io/parsers/c_parser_wrapper.py in read(self, nrows)
    237
    238         else:
--> 239             data = self._reader.read(nrows)
    240         except StopIteration:
    241             if self._first_chunk:

parsers.pyx in pandas._libs.parsers.TextReader.read()

parsers.pyx in pandas._libs.parsers.TextReader._read_rows()

parsers.pyx in pandas._libs.parsers.TextReader._check_tokenize_status()

parsers.pyx in pandas._libs.parsers.raise_parser_error()

ParserError: Error tokenizing data. C error: Expected 19 fields in line 4514, saw 20
```

▼ **PART 3 — about data info **

```
products.head()
```

	product_id	product_name	brand_id	brand_name	loves_count	rating	reviews	size	variation_type	variation_value	...
0	P473671	Fragrance Discovery Set	6342	19-69	6320	3.6364	11.0	NaN	NaN	NaN	...
1	P473668	La Habana Eau de Parfum	6342	19-69	3827	4.1538	13.0	3.4 oz/ 100 mL	Size + Concentration + Formulation	3.4 oz/ 100 mL	...
2	P473662	Rainbow Bar Eau de Parfum	6342	19-69	3253	4.2500	16.0	3.4 oz/ 100 mL	Size + Concentration + Formulation	3.4 oz/ 100 mL	...
3	P473660	Kasbah Eau de Parfum	6342	19-69	3018	4.4762	21.0	3.4 oz/ 100 mL	Size + Concentration + Formulation	3.4 oz/ 100 mL	...
4	P473658	Purple Haze Eau de Parfum	6342	19-69	2691	3.2308	13.0	3.4 oz/ 100 mL	Size + Concentration + Formulation	3.4 oz/ 100 mL	...

5 rows × 27 columns

reviews.head()

	Unnamed: 0	author_id	rating	is_recommended	helpfulness	total_feedback_count	total_neg_feedback_count	total_pos_fee
0	0	1741593524	5	1.0	1.0	2	0	
1	1	31423088263	1	0.0	NaN	0	0	
2	2	5061282401	5	1.0	NaN	0	0	
3	3	6083038851	5	1.0	NaN	0	0	
4	4	47056667835	5	1.0	NaN	0	0	

```
print("products shape:",products.shape)
print("reviews.shape:",reviews.shape)
```

```
products shape: (8494, 27)
reviews.shape: (602130, 19)
```

▼ PART 4 — Understand Columns

products.columns

```
Index(['product_id', 'product_name', 'brand_id', 'brand_name', 'loves_count',
       'rating', 'reviews', 'size', 'variation_type', 'variation_value',
       'variation_desc', 'ingredients', 'price_usd', 'value_price_usd',
       'sale_price_usd', 'limited_edition', 'new', 'online_only',
       'out_of_stock', 'sephora_exclusive', 'highlights', 'primary_category',
       'secondary_category', 'tertiary_category', 'child_count',
```

```
'child_max_price', 'child_min_price'],
dtype='object')
```

```
reviews.columns
```

```
Index(['Unnamed: 0', 'author_id', 'rating', 'is_recommended', 'helpfulness',
      'total_feedback_count', 'total_neg_feedback_count',
      'total_pos_feedback_count', 'submission_time', 'review_text',
      'review_title', 'skin_tone', 'eye_color', 'skin_type', 'hair_color',
      'product_id', 'product_name', 'brand_name', 'price_usd'],
      dtype='object')
```

PART 5 — Data Profiling

```
products.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8494 entries, 0 to 8493
Data columns (total 27 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   product_id            8494 non-null   object
 1   product_name          8494 non-null   object
 2   brand_id              8494 non-null   int64
 3   brand_name            8494 non-null   object
 4   loves_count           8494 non-null   int64
 5   rating                8216 non-null   float64
 6   reviews               8216 non-null   float64
 7   size                  6863 non-null   object
 8   variation_type         7050 non-null   object
 9   variation_value        6896 non-null   object
10   variation_desc         1250 non-null   object
11   ingredients            7549 non-null   object
12   price_usd             8494 non-null   float64
13   value_price_usd        451 non-null    float64
14   sale_price_usd         270 non-null    float64
15   limited_edition        8494 non-null   int64
16   new                   8494 non-null   int64
17   online_only            8494 non-null   int64
18   out_of_stock           8494 non-null   int64
19   sephora_exclusive      8494 non-null   int64
20   highlights             6287 non-null   object
21   primary_category       8494 non-null   object
22   secondary_category      8486 non-null   object
23   tertiary_category       7504 non-null   object
24   child_count            8494 non-null   int64
25   child_max_price        2754 non-null   float64
26   child_min_price        2754 non-null   float64
dtypes: float64(7), int64(8), object(12)
memory usage: 1.7+ MB
```

```
reviews.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 602130 entries, 0 to 602129
Data columns (total 19 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Unnamed: 0            602130 non-null int64
 1   author_id             602130 non-null object
 2   rating                602130 non-null int64
 3   is_recommended        484644 non-null float64
 4   helpfulness           270298 non-null float64
 5   total_feedback_count   602130 non-null int64
 6   total_neg_feedback_count 602130 non-null int64
 7   total_pos_feedback_count 602130 non-null int64
 8   submission_time        602130 non-null object
 9   review_text           601131 non-null object
10   review_title          435119 non-null object
11   skin_tone             496074 non-null object
12   eye_color             463642 non-null object
13   skin_type             527447 non-null object
14   hair_color            461049 non-null object
15   product_id            602130 non-null object
16   product_name          602130 non-null object
17   brand_name            602130 non-null object
18   price_usd             602130 non-null float64
dtypes: float64(3), int64(5), object(11)
memory usage: 87.3+ MB
```

```
products.describe()
```

	brand_id	loves_count	rating	reviews	price_usd	value_price_usd	sale_price_usd	limited_edition	
count	8494.000000	8.494000e+03	8216.000000	8216.000000	8494.000000	451.000000	270.000000	8494.000000	84
mean	5422.440546	2.917957e+04	4.194513	448.545521	51.655595	91.168537	20.207889	0.070285	
std	1709.595957	6.609212e+04	0.516694	1101.982529	53.669234	79.195631	24.327352	0.255642	
min	1063.000000	0.000000e+00	1.000000	1.000000	3.000000	0.000000	1.750000	0.000000	
25%	5333.000000	3.758000e+03	3.981725	26.000000	25.000000	45.000000	8.000000	0.000000	
50%	6157.500000	9.880000e+03	4.289350	122.000000	35.000000	67.000000	14.000000	0.000000	
75%	6328.000000	2.684125e+04	4.530525	418.000000	58.000000	108.500000	25.000000	0.000000	
max	8020.000000	1.401068e+06	5.000000	21281.000000	1900.000000	617.000000	320.000000	1.000000	

```
reviews.describe()
```

	Unnamed: 0	rating	is_recommended	helpfulness	total_feedback_count	total_neg_feedback_count	total_pos_
count	602130.000000	602130.000000	484644.000000	270298.000000	602130.000000	602130.000000	
mean	301064.500000	4.311838	0.837922	0.769722	3.660661	0.765473	
std	173820.10313	1.143692	0.368522	0.319103	25.848330	4.968115	
min	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	
25%	150532.250000	4.000000	1.000000	0.666667	0.000000	0.000000	
50%	301064.500000	5.000000	1.000000	0.944444	0.000000	0.000000	
75%	451596.750000	5.000000	1.000000	1.000000	3.000000	0.000000	
max	602129.000000	5.000000	1.000000	1.000000	5464.000000	573.000000	

PART 6 — Missing Values

```
products.isnull().sum()
```

	0
product_id	0
product_name	0
brand_id	0
brand_name	0
loves_count	0
rating	278
reviews	278
size	1631
variation_type	1444
variation_value	1598
variation_desc	7244
ingredients	945
price_usd	0
value_price_usd	8043
sale_price_usd	8224
limited_edition	0
new	0
online_only	0
out_of_stock	0
sephora_exclusive	0
highlights	2207
primary_category	0
secondary_category	8
tertiary_category	990
child_count	0
child_max_price	5740
child_min_price	5740

dtype: int64

```
reviews.isnull().sum()
```

	0
Unnamed: 0	0
author_id	0
rating	0
is_recommended	117486
helpfulness	331832
total_feedback_count	0
total_neg_feedback_count	0
total_pos_feedback_count	0
submission_time	0
review_text	999
review_title	167011
skin_tone	106056
eye_color	138488
skin_type	74683
hair_color	141081
product_id	0
product_name	0
brand_name	0
price_usd	0

dtype: int64

```
missing_products = pd.DataFrame({
    'Missing Count': products.isnull().sum(),
    'Missing %': products.isnull().mean() * 100
})

missing_products.sort_values(
    'Missing %',
    ascending=False
)
```

	Missing Count	Missing %
sale_price_usd	8224	96.821286
value_price_usd	8043	94.690370
variation_desc	7244	85.283730
child_max_price	5740	67.577113
child_min_price	5740	67.577113
highlights	2207	25.983047
size	1631	19.201789
variation_value	1598	18.813280
variation_type	1444	17.000235
tertiary_category	990	11.655286
ingredients	945	11.125500
rating	278	3.272899
reviews	278	3.272899
secondary_category	8	0.094184
loves_count	0	0.000000
product_id	0	0.000000
brand_name	0	0.000000
product_name	0	0.000000
brand_id	0	0.000000
out_of_stock	0	0.000000
online_only	0	0.000000
new	0	0.000000
limited_edition	0	0.000000
price_usd	0	0.000000
sephora_exclusive	0	0.000000
primary_category	0	0.000000
child_count	0	0.000000

```
missing_reviews = pd.DataFrame({
    'Missing Count': reviews.isnull().sum(),
    'Missing %': reviews.isnull().mean() * 100
})

missing_reviews.sort_values(
    'Missing %',
    ascending=False
)
```

	Missing Count	Missing %
helpfulness	331832	55.109694
review_title	167011	27.736701
hair_color	141081	23.430322
eye_color	138488	22.999684
is_recommended	117486	19.511733
skin_tone	106056	17.613472
skin_type	74683	12.403136
review_text	999	0.165911
rating	0	0.000000
Unnamed: 0	0	0.000000
author_id	0	0.000000
total_pos_feedback_count	0	0.000000
submission_time	0	0.000000
total_feedback_count	0	0.000000
total_neg_feedback_count	0	0.000000
product_id	0	0.000000
product_name	0	0.000000
brand_name	0	0.000000
price_usd	0	0.000000

Start coding or [generate](#) with AI.

handle the important rating and reviews

```
print("Missing rating:", products['rating'].isnull().sum())
print("Missing reviews:", products['reviews'].isnull().sum())
missing_rating_reviews = products[
    products['rating'].isnull() | products['reviews'].isnull()
]

print(missing_rating_reviews[
    ['product_id', 'product_name', 'brand_name', 'rating', 'reviews']
].head(20))
```

```
Missing rating: 278
Missing reviews: 278
product_id      product_name \
42    P471525      Arancia di Capri Candle
60    P473154      Mini Baomint Moisturizing Curl Defining Gel
61    P473153      Mini Baomint Moisturizing Curl Defining Cream
62    P473152      Mini Baomint Leave-In Conditioning Styler
63    P473155      Mini Baomint Moisturizing Shampoo
64    P473156      Mini Baomint Protect + Shine Oil Blend
84    P504777      Perfume Sampler Discovery Set
88    P481056      Mediterranean Honeysuckle Mimosa Eau de Parfum...
121   P503998      Algae Peptide Regenerative Moisturizer
188   P463016      Clinical Densifying Styling Mousse
341   P470001      False Top Lashes
393   P504670      Acqua di Goia Perfume Gift Set
395   P504695      Si Passione Gift Set
412   P474814      Clémentine California Pure Perfume Limited Edi...
488   P500755      Cryo Skin Icing Roller + Bright Eyes Gels Set
524   P504852      Aqua Bomb Everyday Hydration Set
592   P502816      Mascara Power Pair Extreme Lengthening Mascara...
594   P504136      Get REAL Duo Lengthening Mascara Value Set
595   P505033      Mini The POREfessional Package Deal pore prime...
689   P481954      Check Your Bases Vitamin Enriched Face Base Set

brand_name  rating  reviews
42    Acqua di Parma    NaN    NaN
60    adwoa beauty    NaN    NaN
61    adwoa beauty    NaN    NaN
62    adwoa beauty    NaN    NaN
63    adwoa beauty    NaN    NaN
64    adwoa beauty    NaN    NaN
84    AERIN            NaN    NaN
88    AERIN            NaN    NaN
121   Algenist         NaN    NaN
188   ALTERNA Haircare    NaN    NaN
```

341	Anastasia Beverly Hills	NaN	NaN
393	Armani Beauty	NaN	NaN
395	Armani Beauty	NaN	NaN
412	Atelier Cologne	NaN	NaN
488	BeautyBio	NaN	NaN
524	belif	NaN	NaN
592	Benefit Cosmetics	NaN	NaN
594	Benefit Cosmetics	NaN	NaN
595	Benefit Cosmetics	NaN	NaN
689	Bobbi Brown	NaN	NaN

i have checked both are missing in the same products:

```
both_missing = products[
    products['rating'].isnull() &
    products['reviews'].isnull()
]
print("Both rating and reviews missing:", both_missing.shape[0])
```

Both rating and reviews missing: 278

check only rating is missing

```
rating_missing = products[
    products['rating'].isnull() &
    products['reviews'].notnull()
]

print("Rating missing but reviews available:", rating_missing.shape[0])
```

Rating missing but reviews available: 0

check only review is missing

```
reviews_missing = products[
    products['reviews'].isnull() &
    products['rating'].notnull()
]

print("Reviews missing but rating available:", reviews_missing.shape[0])
```

Reviews missing but rating available: 0

Both rating and reviews contains null values for both same products

Keep rating/reviews as NaN

```
products['rating'] = products['rating'].fillna(...)
products['reviews'] = products['reviews'].fillna(...)
```

Create a missing-data flag

```
products['rating_missing'] = products['rating'].isnull()
products['reviews_missing'] = products['reviews'].isnull()
```

```
print(products['rating_missing'].value_counts())
print(products['reviews_missing'].value_counts())
```

```
rating_missing
False      8494
Name: count, dtype: int64
reviews_missing
False      8494
Name: count, dtype: int64
```

```
print(products['rating'].isnull().sum())
print(products['rating'].notnull().sum())
print(products['reviews'].isnull().sum())
print(products['reviews'].notnull().sum())
```

```
0
8494
0
8494
```

```
products[products['rating_missing']][
    ['product_id', 'product_name', 'brand_name', 'price_usd',
     'primary_category', 'secondary_category']
].head(20)
products[products['reviews_missing']][
    ['product_id', 'product_name', 'brand_name', 'price_usd',
     'primary_category', 'secondary_category']
].head(20)
```

```
product_id product_name brand_name price_usd primary_category secondary_category
```

```
print("Missing rating:", products['rating'].isnull().sum())

print("Missing reviews:", products['reviews'].isnull().sum())

print(products[['rating', 'reviews']].isnull().sum())
```

```
Missing rating: 0
Missing reviews: 0
rating      0
reviews     0
dtype: int64
```

```
products.isnull().sum()
```

	0
product_id	0
product_name	0
brand_id	0
brand_name	0
loves_count	0
rating	0
reviews	0
size	1631
variation_type	1444
variation_value	1598
variation_desc	7244
ingredients	945
price_usd	0
value_price_usd	8043
sale_price_usd	8224
limited_edition	0
new	0
online_only	0
out_of_stock	0
sephora_exclusive	0
highlights	2207
primary_category	0
secondary_category	8
tertiary_category	990
child_count	0
child_max_price	5740
child_min_price	5740
rating_missing	0
reviews_missing	0

```
dtype: int64
```

Handle categorical missing values

```
products['secondary_category'] = products['secondary_category'].fillna('Unknown')

products['tertiary_category'] = products['tertiary_category'].fillna('Unknown')
```

```
print(products.isnull().sum())
```

```
product_id          0
product_name        0
brand_id            0
brand_name          0
loves_count         0
rating              0
reviews             0
size                1631
variation_type       1444
variation_value      1598
variation_desc       7244
ingredients          945
price_usd           0
value_price_usd      8043
sale_price_usd       8224
limited_edition       0
new                 0
online_only          0
out_of_stock         0
sephora_exclusive    0
highlights          2207
primary_category     0
secondary_category   0
tertiary_category    0
child_count          0
child_max_price      5740
child_min_price      5740
rating_missing       0
reviews_missing      0
dtype: int64
```

```
products = products.drop(
    columns=['rating_missing', 'reviews_missing'],
    errors='ignore'
)
```

Replacing missing values with NaN

```
products['size']=products['size'].fillna('NaN')
products['variation_type']=products['variation_type'].fillna('NaN')
products['variation_value']=products['variation_value'].fillna('NaN')
products['variation_desc']=products['variation_desc'].fillna('NaN')
products['highlights']=products['highlights'].fillna('NaN')
products['ingredients']=products['ingredients'].fillna('NaN')
products['value_price_usd']=products['value_price_usd'].fillna('NaN')
products['sale_price_usd']=products['sale_price_usd'].fillna('NaN')
products['child_max_price']=products['child_max_price'].fillna('NaN')
products['child_min_price']=products['child_min_price'].fillna('NaN')
```

```
print(products.isnull().sum())
```

```
product_id          0
product_name        0
brand_id            0
brand_name          0
loves_count         0
rating              0
reviews             0
size                0
variation_type       0
variation_value      0
variation_desc       0
ingredients          0
price_usd           0
value_price_usd      8043
sale_price_usd       8224
limited_edition       0
new                 0
online_only          0
out_of_stock         0
sephora_exclusive    0
highlights           0
primary_category     0
secondary_category   0
```

```
tertiary_category    0
child_count          0
child_max_price      5740
child_min_price      5740
value_price_usd      0
sale_price_usd       0
child_max_price      0
child_min_price      0
dtype: int64
```

```
products.isnull().sum().sort_values(ascending=False)
```

```

0
sale_price_usd    8224
value_price_usd   8043
child_max_price   5740
child_min_price   5740
loves_count       0
brand_name        0
product_id        0
reviews           0
rating            0
size              0
variation_type     0
variation_desc     0
variation_value    0
product_name       0
brand_id           0
ingredients         0
price_usd          0
limited_edition     0
new                0
sephora_exclusive  0
highlights         0
online_only        0
out_of_stock       0
secondary_category 0
primary_category   0
child_count        0
tertiary_category  0
value_price_usd    0
sale_price_usd     0
child_max_price    0
child_min_price    0
```

```
missing = products.isnull().sum()
print(missing[missing > 0])
```

```
value_price_usd    8043
sale_price_usd     8224
child_max_price    5740
child_min_price    5740
dtype: int64
```

```
products['value_price_usd ']=products['value_price_usd'].fillna('NaN')
products['sale_price_usd ']=products['sale_price_usd '].fillna('NaN')
products['child_max_price ']=products['child_max_price'].fillna('NaN')
products['child_min_price ']=products['child_min_price'].fillna('NaN')
```

```
missing = products.isnull().sum()
print(missing[missing > 0])
```

```
value_price_usd    8043
sale_price_usd     8224
child_max_price    5740
child_min_price    5740
dtype: int64
```

```
products[['value_price_usd',
          'sale_price_usd',
          'child_max_price',
          'child_min_price']].info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8494 entries, 0 to 8493
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   value_price_usd  451 non-null   float64
1   sale_price_usd   270 non-null   float64
2   child_max_price  2754 non-null  float64
3   child_min_price  2754 non-null  float64
dtypes: float64(4)
memory usage: 265.6 KB
```

```
products[['value_price_usd',
          'sale_price_usd',
          'child_min_price',
          'child_max_price']].describe()
```

	value_price_usd	sale_price_usd	child_min_price	child_max_price
count	451.000000	270.000000	2754.000000	2754.000000
mean	91.168537	20.207889	39.665802	53.792023
std	79.195631	24.327352	38.685720	58.765894
min	0.000000	1.750000	3.000000	3.000000
25%	45.000000	8.000000	19.000000	22.000000
50%	67.000000	14.000000	28.000000	32.000000
75%	108.500000	25.000000	42.000000	59.000000
max	617.000000	320.000000	400.000000	570.000000

```
products[['value_price_usd',
          'sale_price_usd',
          'child_min_price',
          'child_max_price']].head(10)
```

	value_price_usd	sale_price_usd	child_min_price	child_max_price
0	NaN	NaN	NaN	NaN
1	NaN	NaN	30.0	85.0
2	NaN	NaN	30.0	75.0
3	NaN	NaN	30.0	75.0
4	NaN	NaN	30.0	75.0
5	NaN	NaN	NaN	NaN
6	NaN	NaN	NaN	NaN
7	NaN	NaN	30.0	75.0
8	NaN	NaN	30.0	75.0
9	NaN	NaN	NaN	NaN

```
products.isnull().sum()
```

	0
product_id	0
product_name	0
brand_id	0
brand_name	0
loves_count	0
rating	0
reviews	0
size	0
variation_type	0
variation_value	0
variation_desc	0
ingredients	0
price_usd	0
value_price_usd	8043
sale_price_usd	8224
limited_edition	0
new	0
online_only	0
out_of_stock	0
sephora_exclusive	0
highlights	0
primary_category	0
secondary_category	0
tertiary_category	0
child_count	0
child_max_price	5740
child_min_price	5740
value_price_usd	0
sale_price_usd	0
child_max_price	0
child_min_price	0

```
for i, col in enumerate(products.columns):
    print(i, col)
```

```
0 product_id
1 product_name
2 brand_id
3 brand_name
4 loves_count
5 rating
6 reviews
7 size
8 variation_type
9 variation_value
10 variation_desc
11 ingredients
12 price_usd
13 value_price_usd
14 sale_price_usd
15 limited_edition
16 new
17 online_only
18 out_of_stock
19 sephora_exclusive
20 highlights
21 primary_category
22 secondary_category
23 tertiary_category
24 child_count
25 child_max_price
26 child_min_price
27 value_price_usd
```

28 sale_price_usd
29 child_max_price
30 child_min_price

products.columns.value_counts()

	count
product_id	1
product_name	1
brand_id	1
brand_name	1
loves_count	1
rating	1
reviews	1
size	1
variation_type	1
variation_value	1
variation_desc	1
ingredients	1
price_usd	1
value_price_usd	1
sale_price_usd	1
limited_edition	1
new	1
online_only	1
out_of_stock	1
sephora_exclusive	1
highlights	1
primary_category	1
secondary_category	1
tertiary_category	1
child_count	1
child_max_price	1
child_min_price	1
value_price_usd	1
sale_price_usd	1
child_max_price	1
child_min_price	1

products[['value_price_usd', 'sale_price_usd',
 'child_min_price', 'child_max_price']].head(10)

	value_price_usd	sale_price_usd	child_min_price	child_max_price
0	NaN	NaN	NaN	NaN
1	NaN	NaN	30.0	85.0
2	NaN	NaN	30.0	75.0
3	NaN	NaN	30.0	75.0
4	NaN	NaN	30.0	75.0
5	NaN	NaN	NaN	NaN
6	NaN	NaN	NaN	NaN
7	NaN	NaN	30.0	75.0
8	NaN	NaN	30.0	75.0
9	NaN	NaN	NaN	NaN

```
duplicate_cols = products.columns[products.columns.duplicated(keep=False)]
```

```
print(duplicate_cols.tolist())
```

```
[]
```

```
for col in ['value_price_usd', 'sale_price_usd',
            'child_min_price', 'child_max_price']:

    print("\nCOLUMN:", col)
    print(products.loc[:, products.columns == col].head(10))
```

```
COLUMN: value_price_usd
```

```
value_price_usd
0      NaN
1      NaN
2      NaN
3      NaN
4      NaN
5      NaN
6      NaN
7      NaN
8      NaN
9      NaN
```

```
COLUMN: sale_price_usd
```

```
sale_price_usd
0      NaN
1      NaN
2      NaN
3      NaN
4      NaN
5      NaN
6      NaN
7      NaN
8      NaN
9      NaN
```

```
COLUMN: child_min_price
```

```
child_min_price
0      NaN
1    30.0
2    30.0
3    30.0
4    30.0
5      NaN
6      NaN
7    30.0
8    30.0
9      NaN
```

```
COLUMN: child_max_price
```

```
child_max_price
0      NaN
1    85.0
2    75.0
3    75.0
4    75.0
5      NaN
6      NaN
7    75.0
8    75.0
9      NaN
```

```
for col in ['value_price_usd', 'sale_price_usd',
            'child_min_price', 'child_max_price']:
```

```
    combined = products.loc[:, products.columns == col].bfill(axis=1).iloc[:, 0]
```

```
    products = products.loc[:, products.columns != col]
```

```
    products[col] = combined
```

```
print(products.columns.tolist())
```

```
['product_id', 'product_name', 'brand_id', 'brand_name', 'loves_count', 'rating', 'reviews', 'size', 'variation_type', 'vari
```

```
products[['value_price_usd',
            'sale_price_usd',
            'child_min_price',
            'child_max_price']].isnull().sum()
```

```

      0
value_price_usd  8043
sale_price_usd   8224
child_min_price  5740
child_max_price  5740

dtype: int64
```

```
products[['value_price_usd',
           'sale_price_usd',
           'child_min_price',
           'child_max_price']].isnull().sum()
```

```

      0
value_price_usd  8043
sale_price_usd   8224
child_min_price  5740
child_max_price  5740

dtype: int64
```

```
products = products.loc[:, ~products.columns.duplicated()]
reviews = reviews.loc[:, ~reviews.columns.duplicated()]
```

removing the duplicate column

```
products = products.drop(columns=['value_price_usd', 'sale_price_usd',
                                  'child_min_price', 'child_max_price'])
```

```
print(products.columns.tolist())
```

```
['product_id', 'product_name', 'brand_id', 'brand_name', 'loves_count', 'rating', 'reviews', 'size', 'variation_type', 'vari
```

```
products.isnull().sum()
```

	0
product_id	0
product_name	0
brand_id	0
brand_name	0
loves_count	0
rating	0
reviews	0
size	0
variation_type	0
variation_value	0
variation_desc	0
ingredients	0
price_usd	0
limited_edition	0
new	0
online_only	0
out_of_stock	0
sephora_exclusive	0
highlights	0
primary_category	0
secondary_category	0
tertiary_category	0
child_count	0
value_price_usd	0
sale_price_usd	0
child_max_price	0
child_min_price	0

dtype: int64

```
reviews.isnull().sum()
```

	0
Unnamed: 0	0
author_id	0
rating	0
is_recommended	117486
helpfulness	331832
total_feedback_count	0
total_neg_feedback_count	0
total_pos_feedback_count	0
submission_time	0
review_text	999
review_title	167011
skin_tone	106056
eye_color	138488
skin_type	74683
hair_color	141081
product_id	0
product_name	0
brand_name	0
price_usd	0

dtype: int64

removing the unwanted column

```
reviews = reviews.drop(columns=['Unnamed: 0'], errors='ignore')
```

```
print(reviews.columns)
```

```
Index(['author_id', 'rating', 'is_recommended', 'helpfulness',
      'total_feedback_count', 'total_neg_feedback_count',
      'total_pos_feedback_count', 'submission_time', 'review_text',
      'review_title', 'skin_tone', 'eye_color', 'skin_type', 'hair_color',
      'product_id', 'product_name', 'brand_name', 'price_usd'],
      dtype='object')
```

```
reviews['is_recommended '] = reviews['is_recommended'].fillna('NaN')
reviews['helpfulness'] = reviews['helpfulness'].fillna('NaN')
reviews['review_text '] = reviews['review_text'].fillna('NaN')
reviews['review_title '] = reviews['review_title'].fillna('NaN')
reviews['skin_tone '] = reviews['skin_tone'].fillna('NaN')
reviews['eye_color '] = reviews['eye_color'].fillna('NaN')
reviews['skin_type '] = reviews['skin_type'].fillna('NaN')
reviews['hair_color '] = reviews['hair_color'].fillna('NaN')
```

```
reviews.isnull().sum()
```

	0
author_id	0
rating	0
is_recommended	117486
helpfulness	0
total_feedback_count	0
total_neg_feedback_count	0
total_pos_feedback_count	0
submission_time	0
review_text	999
review_title	167011
skin_tone	106056
eye_color	138488
skin_type	74683
hair_color	141081
product_id	0
product_name	0
brand_name	0
price_usd	0
is_recommended	0
review_text	0
review_title	0
skin_tone	0
eye_color	0
skin_type	0
hair_color	0

dtype: int64

reviews.columns

```
Index(['author_id', 'rating', 'is_recommended', 'helpfulness',
      'total_feedback_count', 'total_neg_feedback_count',
      'total_pos_feedback_count', 'submission_time', 'review_text',
      'review_title', 'skin_tone', 'eye_color', 'skin_type', 'hair_color',
      'product_id', 'product_name', 'brand_name', 'price_usd',
      'is_recommended ', 'review_text ', 'review_title ', 'skin_tone ',
      'eye_color ', 'skin_type ', 'hair_color '],
      dtype='object')
```

reviews.columns.tolist()

```
['author_id',
 'rating',
 'is_recommended',
 'helpfulness',
 'total_feedback_count',
 'total_neg_feedback_count',
 'total_pos_feedback_count',
 'submission_time',
 'review_text',
 'review_title',
 'skin_tone',
 'eye_color',
 'skin_type',
 'hair_color',
 'product_id',
 'product_name',
 'brand_name',
 'price_usd',
 'is_recommended ',
 'review_text ',
 'review_title ',
 'skin_tone ',
 'eye_color ',
 'skin_type ',
 'hair_color ']
```

```
cols = [
    'is_recommended',
    'review_text',
    'review_title',
    'skin_tone',
    'eye_color',
    'skin_type',
    'hair_color'
]

# Convert empty strings and spaces to NaN
for col in cols:
    reviews[col] = reviews[col].replace(r'^\s*$', np.nan, regex=True)

# Check
print(reviews[cols].isna().sum())
```

```
is_recommended    117486
review_text         999
review_title      167011
skin_tone         106056
eye_color         138488
skin_type          74683
hair_color        141081
dtype: int64
```

```
print(reviews['is_recommended'].unique())
```

```
[ 1.  0. nan]
```

```
print(reviews['is_recommended'].value_counts(dropna=False))
```

```
is_recommended
1.0      406094
NaN      117486
0.0       78550
Name: count, dtype: int64
```

```
reviews['is_recommended'] = reviews['is_recommended'].astype('Float64')
```

```
print(reviews['is_recommended'].value_counts(dropna=False))
```

```
is_recommended
1.0      406094
<NA>     117486
0.0       78550
Name: count, dtype: Int64
```

```
cols = [
    'is_recommended',
    'review_text',
    'review_title',
    'skin_tone',
    'eye_color',
    'skin_type',
    'hair_color'
]

for col in cols:
    reviews[col] = reviews[col].where(reviews[col].notna(), np.nan)
```

Checking duplicate values

```
print("Duplicate products:", products.duplicated().sum())
```

```
print("Duplicate reviews:", reviews.duplicated().sum())
```

```
Duplicate products: 0
Duplicate reviews: 108
```

Removing the duplicate values

```
products = products.drop_duplicates().reset_index(drop=True)
reviews = reviews.drop_duplicates().reset_index(drop=True)
```

```
print("Products shape:", products.shape)
print("Reviews shape:", reviews.shape)

print("Remaining duplicate products:", products.duplicated().sum())
print("Remaining duplicate reviews:", reviews.duplicated().sum())

Products shape: (8494, 27)
Reviews shape: (602022, 25)
Remaining duplicate products: 0
Remaining duplicate reviews: 0
```

Invalid checking in reviews

1. Check rating

```
print(reviews['rating'].value_counts().sort_index())

rating
1      32899
2      29249
3      44015
4     106934
5     388925
Name: count, dtype: int64
```

2. Check is_recommended

```
print(reviews['is_recommended'].value_counts(dropna=False))

is_recommended
1.0      406025
<NA>     117455
0.0      78542
Name: count, dtype: Int64
```

3. Check helpfulness

```
print(reviews['helpfulness'].describe())

count      602022
unique       2315
top          NaN
freq       331744
Name: helpfulness, dtype: object
```

```
reviews['helpfulness'] = pd.to_numeric(
    reviews['helpfulness'],
    errors='coerce'
)
```

converting values into

```
print(reviews['helpfulness'].dtype)

float64
```

```
print("Values below 0:", (reviews['helpfulness'] < 0).sum())
print("Values above 1:", (reviews['helpfulness'] > 1).sum())

Values below 0: 0
Values above 1: 0
```

Check missing values after conversion

```
print("Missing helpfulness:", reviews['helpfulness'].isna().sum())

Missing helpfulness: 331744
```

```
print("Missing helpfulness:", reviews['helpfulness'].isna().sum())
print("Total reviews:", len(reviews))

Missing helpfulness: 331744
Total reviews: 602022
```

```
print("Below 0:", (reviews['helpfulness'] < 0).sum())
print("Above 1:", (reviews['helpfulness'] > 1).sum())
```

```
Below 0: 0
Above 1: 0
```

```
print(reviews['helpfulness'].describe())
```

```
count    270278.000000
mean         0.769742
std         0.319084
min         0.000000
25%         0.666667
50%         0.944444
75%         1.000000
max         1.000000
Name: helpfulness, dtype: float64
```

```
reviews['helpfulness'] = reviews['helpfulness'].astype('float64')
reviews['helpfulness'] = reviews['helpfulness'].fillna(np.nan)
```

```
print(reviews['helpfulness'].isna().sum())
```

```
331744
```

```
print(reviews['helpfulness'].isnull().sum())
```

```
331744
```

helpfulness are filled as NaN

Feedback counts:

```
feedback_cols = [
    'total_feedback_count',
    'total_neg_feedback_count',
    'total_pos_feedback_count'
]

for col in feedback_cols:
    print(col, "negative values:", (reviews[col] < 0).sum())
```

```
total_feedback_count negative values: 0
total_neg_feedback_count negative values: 0
total_pos_feedback_count negative values: 0
```

Check feedback consistency

1. positive feedback + negative feedback $\leq$ total

feedback

```
invalid_feedback = (
    reviews['total_pos_feedback_count'] +
    reviews['total_neg_feedback_count']
    > reviews['total_feedback_count']
)

print("Invalid feedback rows:", invalid_feedback.sum())
```

```
Invalid feedback rows: 0
```

total_pos_feedback_count+total_neg_feedback_count is never greater than total_feedback_count

price_usd

```
print("Negative prices:", (reviews['price_usd'] < 0).sum())
print("Zero prices:", (reviews['price_usd'] == 0).sum())
```

```
Negative prices: 0
Zero prices: 0
```

No invalid prices.

```
print(reviews['price_usd'].describe())
```

```
count    602022.000000
mean      45.725378
std       35.746008
min        3.000000
25%       23.500000
50%       38.000000
75%       60.000000
max       380.000000
Name: price_usd, dtype: float64
```

In valid checks in Products dataset

Convert rating to numeric

```
products['rating'] = pd.to_numeric(
    products['rating'],
    errors='coerce'
)
```

Check missing values

```
print("Missing ratings:", products['rating'].isna().sum())
```

```
Missing ratings: 278
```

These are Nan values we should not replace with 0 or the mean values because it will effect the Customer Satisfaction Score (CSS)

```
print("Below 1:", (products['rating'] < 1).sum())
print("Above 5:", (products['rating'] > 5).sum())
```

```
Below 1: 0
Above 5: 0
```

Check the rating distribution

```
print(products['rating'].describe())
```

```
count    8216.000000
mean      4.194513
std       0.516694
min       1.000000
25%       3.981725
50%       4.289350
75%       4.530525
max       5.000000
Name: rating, dtype: float64
```

```
print(products['reviews'].describe())
```

```
count      8494
unique     1557
top      Ellipsis
freq       278
Name: reviews, dtype: object
```

```
products['reviews'] = pd.to_numeric(
    products['reviews'],
    errors='coerce'
)
```

```
print("Below 0:", (products['reviews'] < 0).sum())
print("Zero reviews:", (products['reviews'] == 0).sum())
```

```
Below 0: 0
Zero reviews: 0
```

price_usd

```
print(products['price_usd'].describe())
```

```
print("Negative prices:", (products['price_usd'] < 0).sum())
print("Zero prices:", (products['price_usd'] == 0).sum())
```

```
count    8494.000000
mean     51.655595
```

```
std      53.669234
min       3.000000
25%      25.000000
50%      35.000000
75%      58.000000
max     1900.000000
Name: price_usd, dtype: float64
Negative prices: 0
Zero prices: 0
```

value_price_usd

```
products.columns = products.columns.str.strip()
```

```
print(products.columns.tolist())
```

```
['product_id', 'product_name', 'brand_id', 'brand_name', 'loves_count', 'rating', 'reviews', 'size', 'variation_type', 'vari
```

```
product_numeric = [
    'rating',
    'reviews',
    'value_price_usd',
    'sale_price_usd',
    'child_max_price',
    'child_min_price'
]

for col in product_numeric:
    products[col] = pd.to_numeric(products[col], errors='coerce')
```

```
print(products[product_numeric].dtypes)
```

```
rating      float64
reviews     float64
value_price_usd float64
sale_price_usd float64
child_max_price float64
child_min_price float64
dtype: object
```

```
print("Negative value prices:", (products['value_price_usd'] < 0).sum())
print("Zero value prices:", (products['value_price_usd'] == 0).sum())
```

```
Negative value prices: 0
Zero value prices: 2
```

```
products[products['value_price_usd'] == 0][
    ['product_id', 'product_name', 'price_usd', 'value_price_usd']
]
```

	product_id	product_name	price_usd	value_price_usd
569	P500840	Gimme Brow+ Volumizing Fiber Eyebrow Pencil	25.0	0.0
588	P501426	Mini Gimme Brow + Volumizing Fiber Eyebrow Pencil	15.0	0.0

```
products.loc[
    products['value_price_usd'] == 0,
    'value_price_usd'
] = np.nan
```

```
print("Zero values:", (products['value_price_usd'] == 0).sum())
print("Missing values:", products['value_price_usd'].isna().sum())
```

```
Zero values: 0
Missing values: 8045
```

sale_price_usd

```
print(products['sale_price_usd'].describe())
```

```
print("Negative sale prices:", (products['sale_price_usd'] < 0).sum())
print("Zero sale prices:", (products['sale_price_usd'] == 0).sum())
print("Missing sale prices:", products['sale_price_usd'].isna().sum())
```

```
count    270.000000
mean      20.207889
```

```
std      24.327352
min      1.750000
25%      8.000000
50%     14.000000
75%     25.000000
max     320.000000
Name: sale_price_usd, dtype: float64
Negative sale prices: 0
Zero sale prices: 0
Missing sale prices: 8224
```

child_max_price

```
print(products['child_max_price'].describe())

print("Negative values:", (products['child_max_price'] < 0).sum())
print("Zero values:", (products['child_max_price'] == 0).sum())
print("Missing values:", products['child_max_price'].isna().sum())
```

```
count    2754.000000
mean     53.792023
std      58.765894
min       3.000000
25%     22.000000
50%     32.000000
75%     59.000000
max     570.000000
Name: child_max_price, dtype: float64
Negative values: 0
Zero values: 0
Missing values: 5740
```

child_min_price

```
print(products['child_min_price'].describe())

print("Negative values:", (products['child_min_price'] < 0).sum())
print("Zero values:", (products['child_min_price'] == 0).sum())
print("Missing values:", products['child_min_price'].isna().sum())
```

```
count    2754.000000
mean     39.665802
std      38.685720
min       3.000000
25%     19.000000
50%     28.000000
75%     42.000000
max     400.000000
Name: child_min_price, dtype: float64
Negative values: 0
Zero values: 0
Missing values: 5740
```

Check the 0/1 flag columns

```
flag_cols = [
    'limited_edition',
    'new',
    'online_only',
    'out_of_stock',
    'sephora_exclusive'
]

for col in flag_cols:
    print("\n", col)
    print(products[col].value_counts(dropna=False))
```

```
limited_edition
limited_edition
0    7897
1     597
Name: count, dtype: int64
```

```
new
new
0    7885
1     609
Name: count, dtype: int64
```

```
online_only
online_only
0    6633
```

```

1    1861
Name: count, dtype: int64

    out_of_stock
out_of_stock
0    7868
1     626
Name: count, dtype: int64

    sephora_exclusive
sephora_exclusive
0    6121
1    2373
Name: count, dtype: int64

```

Check child_count

```

print(products['child_count'].describe())

print("Negative child counts:",
      (products['child_count'] < 0).sum())

print("Zero child counts:",
      (products['child_count'] == 0).sum())

```

```

count      8494.000000
mean         1.631622
std          5.379470
min          0.000000
25%          0.000000
50%          0.000000
75%          1.000000
max         105.000000
Name: child_count, dtype: float64
Negative child counts: 0
Zero child counts: 5740

```

Zero is valid here — it likely means the product has no child/variant products.

price relationship

```

invalid_child_price = (
    products['child_min_price'] > products['child_max_price']
)

print("Invalid child price rows:", invalid_child_price.sum())

```

```
Invalid child price rows: 0
```

Outlier Detection

```
products['price_usd'].describe()
```

```

      price_usd
count  8494.000000
mean    51.655595
std     53.669234
min      3.000000
25%     25.000000
50%     35.000000
75%     58.000000
max    1900.000000

dtype: float64

```

For outlier detection, use the IQR method. **Step 1 — Calculate IQR**

```

Q1 = products['price_usd'].quantile(0.25)
Q3 = products['price_usd'].quantile(0.75)

IQR = Q3 - Q1

```

```
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
```

```
Q1: 25.0
Q3: 58.0
IQR: 33.0
```

Calculate the upper limit

```
upper_limit = Q3 + 1.5 * IQR

print("Upper limit:", upper_limit)
```

```
Upper limit: 107.5
```

So any price_usd > 107.5 will be identified as a statistical outlier.

Count the outliers

```
outliers = products[products['price_usd'] > upper_limit]

print("Number of price outliers:", len(outliers))
```

```
Number of price outliers: 845
```

A high-priced Sephora product can be a genuine product, so an outlier is not automatically an error.

```
price_outliers = products[
    products['price_usd'] > upper_limit
]

price_outliers[
    ['product_id', 'product_name', 'brand_name',
     'price_usd', 'primary_category']
].sort_values('price_usd', ascending=False).head(20)
```

	product_id	product_name	brand_name	price_usd	primary_category
6802	P502216	Shani Darden by Déesse PRO LED Light Mask	Shani Darden Skin Care	1900.0	Skincare
5556	P504676	NuFACE Trinity+ & Effective Lip and Eye Attach...	NuFACE	495.0	Skincare
2162	P471545	DRx SpectraLite BodyWare Pro	Dr. Dennis Gross Skincare	465.0	Skincare
3510	P395417	Touch Home Permanent Hair Removal Device	iluminage	449.0	Skincare
5551	P393600	Trinity Facial Toning Device + Wrinkle Reducer...	NuFACE	449.0	Skincare
5545	P385320	Trinity + Eye and Lip Enhancer Attachment Bundle	NuFACE	449.0	Skincare
5546	P402992	Trinity + Eye and Lip Enhancer Attachment Bundle	NuFACE	429.0	Skincare
5553	P402993	Trinity Facial Toning Device + Wrinkle Reducer...	NuFACE	429.0	Skincare
4313	P461949	The Concentrate Serum	La Mer	425.0	Skincare
4320	P450290	The Regenerating Serum	La Mer	410.0	Skincare
6937	P375854	Ultimate Revival Cream	SK-II	400.0	Skincare
5554	P500464	NuBODY - Body Toning Device	NuFACE	399.0	Skincare
2835	P504606	Duet Style 2-in-1 Hot Air Styler	ghd	399.0	Hair
5548	P428005	NuBODY Skin Toning Device	NuFACE	399.0	Skincare
6799	P471097	Facial Sculpting Wand	Shani Darden Skin Care	399.0	Skincare
7769	P455598	Rose Prick	TOM FORD	395.0	Fragrance
7811	P468710	Tubéreuse Nue	TOM FORD	395.0	Fragrance
7808	P470554	Soleil Brulant	TOM FORD	395.0	Fragrance
7760	P429291	Fucking Fabulous	TOM FORD	395.0	Fragrance
7758	P464304	Bitter Peach Eau De Parfum	TOM FORD	395.0	Fragrance

```
products.nlargest(10, 'price_usd')[
    ['product_id', 'product_name', 'brand_name', 'price_usd']
]
```

	product_id	product_name	brand_name	price_usd
6802	P502216	Shani Darden by Déesse PRO LED Light Mask	Shani Darden Skin Care	1900.0
5556	P504676	NuFACE Trinity+ & Effective Lip and Eye Attach...	NuFACE	495.0
2162	P471545	DRx SpectraLite BodyWare Pro	Dr. Dennis Gross Skincare	465.0
3510	P395417	Touch Home Permanent Hair Removal Device	iluminage	449.0
5545	P385320	Trinity + Eye and Lip Enhancer Attachment Bundle	NuFACE	449.0
5551	P393600	Trinity Facial Toning Device + Wrinkle Reducer...	NuFACE	449.0
5546	P402992	Trinity + Eye and Lip Enhancer Attachment Bundle	NuFACE	429.0
5553	P402993	Trinity Facial Toning Device + Wrinkle Reducer...	NuFACE	429.0
4313	P461949	The Concentrate Serum	La Mer	425.0
4320	P450290	The Regenerating Serum	La Mer	410.0

IQR upper limit: 107.50 *Number of statistical outliers* : 845 *Maximum price* :1,900 Treatment: Retained

Price outliers were identified using the IQR method. Although 845 products exceeded the upper limit of \$107.50, inspection of the highest-priced products showed that these were genuine premium and luxury products. Therefore, the outliers were retained to preserve the original business information.

loves_count outliers

```
Q1 = products['loves_count'].quantile(0.25)
Q3 = products['loves_count'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

loves_outliers = products[
    (products['loves_count'] < lower_limit) |
    (products['loves_count'] > upper_limit)
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower limit:", lower_limit)
print("Upper limit:", upper_limit)
print("Number of loves_count outliers:", len(loves_outliers))
```

```
Q1: 3758.0
Q3: 26841.25
IQR: 23083.25
Lower limit: -30866.875
Upper limit: 61466.125
Number of loves_count outliers: 951
```

Since loves_count cannot realistically be negative, the important condition is:

$\text{loves_count} > 61,466.125$

These are products with unusually high customer engagement. They may be popular products, not incorrect data.

```
loves_outliers = products[
    products['loves_count'] > upper_limit
]

loves_outliers[
    ['product_id', 'product_name', 'brand_name', 'loves_count', 'rating']
].sort_values('loves_count', ascending=False).head(20)
```

	product_id	product_name	brand_name	loves_count	rating
6242	P97989778	Soft Pinch Liquid Blush	Rare Beauty by Selena Gomez	1401068	4.5356
5249	P377873	Radiant Creamy Concealer	NARS	1153594	4.3080
4431	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	LANEIGE	1081315	4.3508
6434	P281411	Cream Lip Stain Liquid Lipstick	SEPHORA COLLECTION	1029051	4.3201
2523	P67988453	Gloss Bomb Universal Lip Luminizer	Fenty Beauty by Rihanna	968317	4.6357
2524	P87985432	Pro Filt'r Soft Matte Longwear Liquid Foundation	Fenty Beauty by Rihanna	856497	4.0356
5250	P2855	Blush	NARS	840076	4.6643
247	P202633	Brow Wiz Ultra-Slim Precision Brow Pencil	Anastasia Beverly Hills	834189	4.4056
4458	P109908	Translucent Loose Setting Powder	Laura Mercier	813497	4.5029
7648	P427417	Niacinamide 10% + Zinc 1% Oil Control Serum	The Ordinary	763168	4.2439
1893	P236816	Dior Addict Lip Glow	Dior	757716	4.3517
7651	P427419	Hyaluronic Acid 2% + B5 Hydrating Serum	The Ordinary	720504	4.2133
1894	P453814	Lip Glow Oil	Dior	684860	4.0795
7976	P263504	All Nighter Long-Lasting Makeup Setting Spray	Urban Decay	652806	4.4470
2402	P378284	Double Wear Stay-in-Place Foundation	Estée Lauder	639063	4.3756
343	P393401	Luminous Silk Perfect Glow Flawless Oil-Free F...	Armani Beauty	622995	4.2162
248	P384060	DIPBROW Waterproof, Smudge Proof Brow Pomade	Anastasia Beverly Hills	617437	4.6701
249	P404832	Eye Shadow Singles	Anastasia Beverly Hills	605038	4.4696
7977	P409523	Vice Hydrating Lipstick	Urban Decay	593515	4.5825
250	P404831	Liquid Lipstick	Anastasia Beverly Hills	583724	3.8268

Loves Count Outliers: Using the IQR method, 951 products were identified as outliers with loves_count above 61,466.125. Inspection showed that these products represent highly popular products with legitimately high customer engagement. Therefore, the outliers were retained

child_count

```
Q1 = products['child_count'].quantile(0.25)
Q3 = products['child_count'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

child_outliers = products[
    (products['child_count'] < lower_limit) |
    (products['child_count'] > upper_limit)
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower limit:", lower_limit)
print("Upper limit:", upper_limit)
print("Number of child_count outliers:", len(child_outliers))
```

```
Q1: 0.0
Q3: 1.0
IQR: 1.0
Lower limit: -1.5
Upper limit: 2.5
Number of child_count outliers: 1101
```

Because child_count represents the number of variants, values greater than 2.5 mean products with 3 or more variants. That can be completely normal for Sephora products.

```
child_outliers = products[
    products['child_count'] > upper_limit
]

child_outliers[
    ['product_id', 'product_name', 'brand_name',
     'child_count', 'price_usd']
].sort_values('child_count', ascending=False).head(20)
```

	product_id	product_name	brand_name	child_count	price_usd
2524	P87985432	Pro Filt'r Soft Matte Longwear Liquid Foundation	Fenty Beauty by Rihanna	105	40.0
343	P393401	Luminous Silk Perfect Glow Flawless Oil-Free F...	Armani Beauty	78	69.0
6435	P430932	Sephora Colorful Eyeshadow	SEPHORA COLLECTION	73	10.0
6434	P281411	Cream Lip Stain Liquid Lipstick	SEPHORA COLLECTION	73	15.0
7385	P481746	SEA Power Flex Full Coverage Vegan Concealer	tarte	63	28.0
5249	P377873	Radiant Creamy Concealer	NARS	59	32.0
7347	P433206	Maracuja Creaseless Concealer	tarte	59	29.0
2402	P378284	Double Wear Stay-in-Place Foundation	Estée Lauder	55	48.0
7414	P473821	Amazonian Clay 16-Hour Full Coverage Foundation	tarte	51	40.0
3150	P502185	Triclone Skin Tech Medium Coverage Foundation ...	HAUS LABS BY LADY GAGA	50	45.0
1406	P234967	Even Better Makeup Broad Spectrum SPF 15 Found...	CLINIQUE	50	34.0
6451	P467140	Best Skin Ever Liquid Foundation	SEPHORA COLLECTION	49	20.0
8039	P479501	Rosso Valentino High Pigment Refillable Lipstick	Valentino	49	45.0
6476	P500453	Best Skin Ever Full Coverage Multi-Use Concealer	SEPHORA COLLECTION	49	15.0
272	P449124	Luminous Foundation	Anastasia Beverly Hills	49	38.0
2575	P476525	Mini Pro Filt'r Soft Matte Foundation	Fenty Beauty by Rihanna	49	15.0
2531	P88779809	Pro Filt'r Instant Retouch Longwear Liquid Con...	Fenty Beauty by Rihanna	49	30.0
3802	P434015	Essential High Coverage Crème Foundation	Jouer Cosmetics	49	38.0
2547	P466829	Pro Filt'r Soft Matte Longwear Powder Foundation	Fenty Beauty by Rihanna	49	40.0
6246	P64546856	Liquid Touch Brightening Concealer	Rare Beauty by Selena Gomez	47	22.0

Using the IQR method, 1,101 products were identified as outliers with child_count above 2.5. Inspection showed that these products were mainly products with multiple shade or variant options. Therefore, the outliers were considered legitimate business values and retained.

Reviews outliers

total_feedback_count.

```
Q1 = reviews['total_feedback_count'].quantile(0.25)
Q3 = reviews['total_feedback_count'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
```

```

upper_limit = Q3 + 1.5 * IQR

feedback_outliers = reviews[
    (reviews['total_feedback_count'] < lower_limit) |
    (reviews['total_feedback_count'] > upper_limit)
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower limit:", lower_limit)
print("Upper limit:", upper_limit)
print("Number of feedback outliers:", len(feedback_outliers))

```

```

Q1: 0.0
Q3: 3.0
IQR: 3.0
Lower limit: -4.5
Upper limit: 7.5
Number of feedback outliers: 65963

```

```

feedback_outliers = reviews[
    reviews['total_feedback_count'] > upper_limit
]

feedback_outliers[
    ['product_id', 'product_name', 'rating',
     'total_feedback_count',
     'total_pos_feedback_count',
     'total_neg_feedback_count']
].sort_values(
    'total_feedback_count',
    ascending=False
).head(20)

```

	product_id	product_name	rating	total_feedback_count	total_pos_feedback_count	total_neg_feedback_count
11864	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	5	5464	5050	414
15127	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	1	5119	4924	195
90682	P427417	Niacinamide 10% + Zinc 1% Oil Control Serum	5	3851	3481	370
261836	P439061	Glow2OH Dark Spot Toner	3	3458	3110	348
225535	P442563	AHA 30% + BHA 2% Exfoliating Peeling Solution	5	3360	2999	361
536504	P416341	Crème de la Mer Moisturizer	5	2946	2660	286
537882	P426501	Mini Crème de la Mer Moisturizer	5	2946	2660	286
279553	P431180	Self Tanning Bronzing Face Drops	5	2598	2414	184
168371	P454794	Mini The Water Cream	5	2505	2309	196
164202	P418218	The Water Cream Oil- Free Pore Minimizing Moist...	5	2505	2309	196
60710	P417238	Green Clean Makeup Removing Cleansing Balm	1	2304	1977	327
66877	P450271	Green Clean Makeup Meltaway Cleansing Balm Lim...	1	2304	1977	327
225554	P442563	AHA 30% + BHA 2% Exfoliating Peeling Solution	5	2144	1896	248
481434	P427420	Multi-Peptide + HA Serum	5	1982	1814	168

The IQR method identified 65,963 statistical outliers above 7.5 feedback responses. These observations were further validated for negative values and consistency between total, positive, and negative feedback counts. No invalid negative values or inconsistencies were found. Therefore, the extreme values were retained as potentially valid customer-engagement observations.

helpfulness

```
Q1 = reviews['helpfulness'].quantile(0.25)
Q3 = reviews['helpfulness'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

helpfulness_outliers = reviews[
    (reviews['helpfulness'] < lower_limit) |
    (reviews['helpfulness'] > upper_limit)
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower limit:", lower_limit)
print("Upper limit:", upper_limit)
print("Number of helpfulness outliers:", len(helpfulness_outliers))
```

```
Q1: 0.6666669845581055
Q3: 1.0
IQR: 0.33333301544189453
Lower limit: 0.16666746139526367
Upper limit: 1.4999995231628418
Number of helpfulness outliers: 26827
```

```
helpfulness_outliers = reviews[
    reviews['helpfulness'] < lower_limit
]

helpfulness_outliers[
    ['product_id', 'product_name', 'rating', 'helpfulness',
     'total_feedback_count',
     'total_pos_feedback_count',
     'total_neg_feedback_count']
].head(20)
```

	product_id	product_name	rating	helpfulness	total_feedback_count	total_pos_feedback_count	total_neg_feedback_coun
20	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	2	0.0	1	0	
32	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	2	0.0	1	0	
45	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	3	0.0	1	0	
50	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	5	0.0	1	0	
70	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	3	0.0	1	0	
84	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	5	0.0	1	0	
95	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	5	0.0	1	0	
103	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	3	0.0	1	0	
130	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	5	0.0	1	0	
134	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	5	0.0	1	0	
159	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	4	0.0	1	0	
167	P420652	Lip Sleeping Mask Intense Hydration with	3	0.0	4	0	

The IQR method identified 26,827 statistical outliers below the lower threshold of 0.1667. However, all observed helpfulness values remained within the valid range of 0–1. These observations were therefore treated as valid low-helpfulness values and retained rather than removed.

The values can ≥ 0 and ≤ 1 for valid values and min&max values.

```
print("Minimum helpfulness:", reviews['helpfulness'].min())
print("Maximum helpfulness:", reviews['helpfulness'].max())

invalid_helpfulness = reviews[
    (reviews['helpfulness'] < 0) |
    (reviews['helpfulness'] > 1)
]

print("Invalid helpfulness values:", len(invalid_helpfulness))

Minimum helpfulness: 0.0
Maximum helpfulness: 1.0
Invalid helpfulness values: 0
```

all helpfulness values are between 0 and 1

total_pos_feedback_count

```
Q1 = reviews['total_pos_feedback_count'].quantile(0.25)
Q3 = reviews['total_pos_feedback_count'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

pos_outliers = reviews[
    (reviews['total_pos_feedback_count'] < lower_limit) |
    (reviews['total_pos_feedback_count'] > upper_limit)
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower limit:", lower_limit)
print("Upper limit:", upper_limit)
print("Number of positive feedback outliers:", len(pos_outliers))
```

```
Q1: 0.0
Q3: 2.0
IQR: 2.0
Lower limit: -3.0
Upper limit: 5.0
Number of positive feedback outliers: 70872
```

```
pos_outliers = reviews[
    reviews['total_pos_feedback_count'] > upper_limit
]

pos_outliers[
    ['product_id', 'product_name', 'rating',
     'total_feedback_count',
     'total_pos_feedback_count',
     'total_neg_feedback_count']
].sort_values(
    'total_pos_feedback_count',
    ascending=False
).head(20)
```

	product_id	product_name	rating	total_feedback_count	total_pos_feedback_count	total_neg_feedback_count
11864	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	5	5464	5050	414
15127	P420652	Lip Sleeping Mask Intense Hydration with Vitam...	1	5119	4924	195

Niacinamide 10% +

```
invalid_pos = reviews[
    (reviews['total_pos_feedback_count'] < 0) |
    (reviews['total_pos_feedback_count'] > reviews['total_feedback_count'])
]

print("Invalid positive feedback rows:", len(invalid_pos))
```

Invalid positive feedback rows: 0

537882	P426501	Mini Crème de la Mer Moisturizer	5	2946	2660	286
536504	P416341	Moisturizer	5	2946	2660	286

The IQR method identified 70,872 reviews with more than 5 positive-feedback responses. The observations were validated to ensure that feedback counts were non-negative and that positive feedback did not exceed total feedback. No logical inconsistencies were identified; therefore, these observations were retained as valid engagement data.

279553	P431180	Self Tanning Bronzing Face Drops	5	2598	2414	184
total_neg_feedback_count						

101000	P410010	The Water Cream Oil-	5	2505	2300	205
--------	---------	----------------------	---	------	------	-----

```
Q1 = reviews['total_neg_feedback_count'].quantile(0.25)
Q3 = reviews['total_neg_feedback_count'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

neg_outliers = reviews[
    (reviews['total_neg_feedback_count'] < lower_limit) |
    (reviews['total_neg_feedback_count'] > upper_limit)
]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower limit:", lower_limit)
print("Upper limit:", upper_limit)
print("Number of negative feedback outliers:", len(neg_outliers))
```